Cheatsheet: Gerenciamento de Memória

ARC (Swift) vs Garbage Collection / LeakCanary (Android)

Guia Técnico

Atenção: Memory Leaks resultam em *OutOfMemoryError (OOM)* no Android e encerramento sumário do processo via *Jetsam* no iOS. Identificar reter referências cíclicas é vital.

1. SWIFT ARC: WEAK VS UNOWNED

SWIFT WEAK

```
// Opcional, torna-se nil se desalocado.
class Service {
  weak var delegate: ServiceDelegate?
}

// Previne Retain Cycle em Closures:
network.fetch { [weak self] result in
  guard let self = self else { return }
  self.process(result)
}
```

SWIFT UNOWNED

```
// Não-opcional, presume vida igual/maior.
class CreditCard {
  unowned let owner: Customer
  init(owner: Customer) {
    self.owner = owner
  }
}
// ⚠️ Se owner for desalocado: CRASH!
```

2. COMPARATIVO TECNOLÓGICO DE MEMÓRIA

Conceito	iOS (Swift)	Android (Kotlin)
Mecanismo Base	ARC (Automatic Reference Counting - Tempo de Compilação)	Garbage Collector (ART Runtime - Mark & Sweep)
Ciclos de Retenção	Frequentes em Closures/Delegates (Requer `weak`)	Comum em Inner Classes, Static Views, Async Observers
Ferramenta Principal	Xcode Memory Graph Debugger / Instruments (Leaks)	Android Studio Profiler / LeakCanary Library
Sinal de Alerta SO	`didReceiveMemoryWarning` / Jetsam Termination	`onLowMemory()` / `onTrimMemory()`

3. KOTLIN: PREVENÇÃO DE LEAKS NO ANDROID

KOTLIN ANTI-PATTERN

```
// ❌ Static reference vaza Context/Activity
object DataHolder {
  var context: Context? = null // LEAK!
}

// ❌ View Listener sem limpeza
viewTreeObserver.addOnGlobalLayoutListener {
  // Mantém referência implícita do Fragment
}
```

KOTLIN CORREÇÃO

```
// ✅ Use ApplicationContext se necessário
object DataHolder {
  fun init(app: Application) { ... }
}

// ✅ Clear ViewBinding em Fragments
override fun onDestroyView() {
  super.onDestroyView()
  _binding = null // Vital para GC!
}
```

4. ROTEIRO DE PROFILING E DIAGNÓSTICO

Etapa	Procedimento no Xcode (iOS)	Procedimento no Android Studio
1. Detecção	Ativar *Main Thread Checker* e monitorar o painel Memory no Debug Navigator.	Adicionar a biblioteca `LeakCanary` em escopo `debugImplementation`.
2. Captura	Clicar em *Debug Memory Graph* para congelar a hierarquia de objetos.	Clicar em *Dump Java Heap* na aba Memory Profiler.
3. Análise	Procurar ícones roxos de exclamação que indicam Retain Cycles.	Filtrar por "Leaked Activities/Fragments" na árvore de referências GC Root.

5. REGRAS DE OURO PARA ENGENHEIROS MOBILE

- **Em Swift:** Sempre use `[weak self]` em closures de retornos assíncronos ou que pertençam a objetos de vida mais longa.
- **Em Android:** NUNCA armazene instâncias de `Activity`, `Fragment` ou `View` em variáveis `companion object` ou Singletons.
- **Em Compose/SwiftUI:** Garanta que efeitos colaterais como `LaunchedEffect` ou `.task` sejam cancelados automaticamente quando a View sair da tela.